

PRAKTIKUM SISTEM OPERASI

MODUL 11

(Memori Xinu)



Oleh:

(Ammar Dzaki Nandana)

(2311104071)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

MODUL 11

1. Freememory

a. Buat file source code xsh_freememory.c

Masuk ke xinu/shell, buat xsh_freememory.c

```
include <xinu.h>
include <stdio.h>
include "shprototypes.h"

*****/
* Table of Xinu shell commands and the function associated with each */
*****/
const struct cmdent cmdtab[] = {
    {"argecho", xsh_argecho},
    {"freememory", xsh_freememory},
    {"arp", xsh_arp},
    {"cat", xsh_cat},
    {"clear", xsh_clear},
    {"date", xsh_date},
    {"devdump", xsh_devdump},
    {"echo", xsh_echo},
    {"help", xsh_help},
    {"ls", xsh_ls},
    {"kill", xsh_kill},
    {"memdump", xsh_memdump},
    {"memstat", xsh_memstat},
    {"ns", xsh_ns},
    {"netinfo", xsh_netinfo},
    {"ping", xsh_ping},
    {"ps", xsh_ps},
    {"sleep", xsh_sleep},

```

b. Daftarkan command di shell.cc. Tambahkan deklarasi di shprototypes.h

```
Open [F] Save
~/xinu/include

/* in file xsh_argecho.c */
extern shellcmd xsh_argecho (int32, char *[]);

extern shellcmd xsh_freememory (int32, char *[]);

/* in file xsh_arp.c */
extern shellcmd xsh_arp (int32, char *[]);

/* in file xsh_bingid.c */
extern shellcmd xsh_bingid (int32, char *[]);

/* in file xsh_cat.c */
extern shellcmd xsh_cat (int32, char *[]);

/* in file xsh_clear.c */
extern shellcmd xsh_clear (int32, char *[]);

/* in file xsh_date.c */
extern shellcmd xsh_date (int32, char *[]);

/* in file xsh_devdump.c */
extern shellcmd xsh_devdump (int32, char *[]);

/* in file xsh_echo.c */
extern shellcmd xsh_echo (int32, char *[]);

/* in file xsh_ethstat.c */
extern shellcmd xsh_ethstat (int32, char *[]);
```

d. Compile

Masuk ke ~/xinu/compile, jalankan make lalu make

e. Testing output

```

Welcome to Xinu!

xsh $ ?

shell commands are:

argecho      date      kill      ping      udpecho
freememory   devdump  memdump   ps        udpeserver
arp          echo      memstat   sleep     uptime
cat          help      ns        tee       mycmd
clear        ls        netinfo   udp       ?

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018e110    -975128      0xffff11ee8
0x00100000     5070848      0x004d6000
0x005e8000      57344        0x0000e000
Total FreeSpace  4153064
xsh $

```

1. Implementasi Perintah **freememory**

Pada praktikum ini dilakukan pembuatan command baru pada shell Xinu dengan nama **freememory**. Langkah pertama yang dilakukan adalah membuat file source code **xsh_freememory.c** pada direktori **xinu/shell**. File ini digunakan untuk menambahkan fungsi command baru yang dapat dijalankan melalui terminal Xinu untuk menampilkan informasi memori bebas yang tersedia pada sistem.

Setelah file source code dibuat, command tersebut didaftarkan pada file **shell.c** agar shell Xinu dapat mengenali dan menjalankan perintah **freememory**. Selain itu, deklarasi fungsi juga ditambahkan pada file **shprototypes.h** supaya fungsi command dapat dikenali oleh compiler ketika proses build dilakukan.

Tahap berikutnya adalah melakukan proses compile pada Xinu dengan masuk ke direktori **~/xinu/compile**, kemudian menjalankan perintah **make clean** dan **make**. Setelah proses compile selesai tanpa error, command **freememory** dapat diuji melalui shell Xinu untuk memastikan output berjalan dengan baik dan informasi memori bebas berhasil ditampilkan.

2. Jawaban Teori

a. Alasan Xinu Memisahkan Data Segment dan BSS Segment

Xinu memisahkan data segment dan BSS segment untuk meningkatkan efisiensi penggunaan memori serta mempermudah pengelolaan program saat dijalankan. Data segment digunakan untuk menyimpan variabel global atau variabel static yang sudah memiliki nilai awal, misalnya **int**

`angka = 10;`. Sementara itu, BSS segment digunakan untuk menyimpan variabel global atau static yang belum diinisialisasi, misalnya `int angka;`.

Pemisahan ini membuat ukuran file executable menjadi lebih kecil karena variabel pada BSS tidak perlu disimpan nilainya di dalam file program. Ketika program dijalankan, sistem hanya akan mengalokasikan memori untuk variabel tersebut dan otomatis mengisinya dengan nilai nol.

b. Pengaruh Alokasi dan Dealokasi Memori terhadap Free Space

Selama program berjalan, proses alokasi dan dealokasi memori sangat memengaruhi ukuran free space pada sistem. Ketika program melakukan alokasi memori menggunakan fungsi seperti `malloc` atau `getmem`, ukuran free space akan berkurang karena sebagian memori digunakan oleh proses.

Sebaliknya, ketika memori dikembalikan menggunakan `free` atau `freemem`, ukuran free space akan bertambah kembali. Namun, penggunaan memori secara terus-menerus dengan ukuran blok yang berbeda-beda dapat menyebabkan kondisi memori menjadi tidak teratur.

Hal ini dapat menimbulkan fragmentasi memori, yaitu kondisi ketika ruang kosong memori terpecah menjadi banyak blok kecil yang terpisah. Akibatnya, meskipun total free space masih besar, sistem belum tentu dapat menyediakan blok memori besar yang berkesinambungan untuk proses baru.

c. Alasan Heap Lebih Berpotensi Menimbulkan Masalah dibandingkan Stack

Heap lebih berpotensi menimbulkan masalah karena pengelolaannya dilakukan secara dinamis oleh programmer atau sistem operasi. Kesalahan kecil dalam pengelolaan heap dapat menyebabkan berbagai masalah pada program.

Beberapa masalah yang sering terjadi pada heap antara lain memory leak karena memori lupa dibebaskan, fragmentasi memori akibat banyak blok kecil yang terpisah, double free karena memori dibebaskan lebih dari satu kali, serta invalid access akibat mengakses memori yang sebenarnya sudah tidak valid.

Berbeda dengan heap, stack memiliki mekanisme pengelolaan yang lebih aman dan terstruktur. Pada stack, proses alokasi dan dealokasi dilakukan secara otomatis menggunakan mekanisme LIFO (*Last In First Out*). Oleh karena itu, stack biasanya lebih cepat, lebih teratur, dan lebih kecil kemungkinan mengalami kesalahan pengelolaan memori.

d. Alasan Xinu Menggunakan Linked List untuk Menyimpan Free Block

Xinu menggunakan struktur data linked list untuk menyimpan free block karena struktur ini lebih fleksibel dalam mengelola blok memori yang ukurannya dapat berubah-ubah secara dinamis.

Dengan linked list, sistem dapat dengan mudah menambahkan atau menghapus blok memori bebas tanpa harus menggunakan ukuran array tetap. Selain itu, linked list juga mempermudah proses penggabungan blok-blok memori yang berdekatan sehingga fragmentasi dapat dikurangi.

Setiap node pada linked list biasanya menyimpan informasi berupa alamat blok memori, ukuran blok, dan pointer menuju blok berikutnya. Struktur ini sangat cocok digunakan pada sistem operasi sederhana seperti Xinu karena lebih efisien dan mudah dikelola.

e. Tantangan Utama Penggunaan Heap di Xinu

Tantangan utama penggunaan heap pada Xinu adalah bagaimana mengelola memori dinamis agar tetap efisien dan tidak mengalami kerusakan data maupun fragmentasi memori.

Salah satu tantangan terbesar adalah fragmentasi memori, yaitu ketika banyak blok kecil tersebar sehingga memori sulit digunakan untuk kebutuhan alokasi besar. Selain itu, terdapat risiko memory leak apabila memori tidak dikembalikan setelah selesai digunakan.

Kesalahan lain yang dapat terjadi adalah double free atau penggunaan pointer yang tidak valid. Kondisi ini dapat menyebabkan sistem menjadi tidak stabil bahkan mengalami crash.

Karena Xinu umumnya berjalan pada perangkat dengan resource terbatas, pengelolaan heap harus dilakukan dengan sangat hati-hati agar penggunaan memori tetap optimal. Selain itu, jika banyak proses mengakses heap secara bersamaan, diperlukan mekanisme sinkronisasi untuk mencegah kerusakan data pada struktur heap.

